

“스케이트 연습” 문제 풀이

작성자: 오주원

서술의 편의를 위해 시작 지점, 도착 지점 대신 0번 지점, $N+1$ 번 지점이라고 하고, $V_0 = V_{N+1} = 0$ 이라고 하자. 또한, 각 부분문제에서의 V 의 상한을 K 라고 하자.

부분문제 1

1만큼만 낮추는 경우를 고려하지 않고 N 개의 지점에서 가질 수 있는 속력은 $O(K^N)$ 가지가 있을 수 있다. 찾은 $O(K^N)$ 가지 방법을 $O(N)$ 으로 가능한지 판별해주고, 가능한 방법 중 속력의 합이 최대인 경우를 출력한다. 따라서, 전체 문제를 $O(K^N \times N)$ 에 해결할 수 있다.

부분문제 2

다음과 같이 $DP\ D[i][j]$ ($0 \leq i \leq N+1, 0 \leq j \leq K$)를 정의하자.

- $D[i][j] := i$ 번 지점에서 j 의 속력을 가졌을 때 가능한 1번 지점부터 i 번 지점까지의 최대 속력 합

i 번째 지점에서 j 의 속력을 만들기 위해선 이전 지점에서 0 이상 $V_i + 1$ 이하의 속력을 가지고 있어야 하므로 다음과 같이 점화식을 세울 수 있다.

- $D[0][0] = 0$
- $D[i][j] = \max_{0 \leq k \leq j+1} \{D[i-1][k] + j\}$ ($1 \leq i \leq N+1, 0 \leq j \leq V_i$)

$N+1$ 번째 지점에서 0의 속력으로 끝나야 하므로 답은 $D[N+1][0]$ 에 들어 있다. 중간 지점에서 속력이 0이 되는 것보다 1이 되는 것이 항상 이득이므로 0이 되는 경우는 따로 처리해주지 않아도 된다. 따라서, 문제를 $O(NK^2)$ 에 해결할 수 있다.

부분문제 3

부분문제 2의 풀이를 가져오자. 여기에 추가로 $DP\ P[i][j]$ ($0 \leq i \leq N+1, 0 \leq j \leq K+1$)를 정의하자.

- $P[i][j] := i$ 번째 지점에서 0이상 j 이하의 속력을 가졌을 때 가능한 1번 지점부터 i 번 지점까지의 최대 속력 합

부분문제 2에서 $D[i][*]$ 의 값을 구한 후, $D[i+1][*]$ 의 값을 구하기 전에 다음과 같이 $P[i][*]$ 를 구한다.

- $P[i][0] = D[i][0]$
- $P[i][j] = \max(P[i][j-1], D[i][j])$ ($1 \leq j \leq V_{i+1} + 1$)

$P[i-1][*]$ 을 구했다면, 다음과 같이 $D[i][*]$ 의 점화식을 세울 수 있다.

- $D[i][j] = P[i-1][j+1] + j$ ($1 \leq j \leq V_i$)

부분문제 2와 D 의 정의는 바뀌지 않았으므로 $D[N+1][0]$ 에 문제의 답이 들어있다. $D[*][*]$ 와 $P[*][*]$ 의 모든 값은 각각 $O(NK)$ 에 구할 수 있으므로, 문제를 $O(NK)$ 에 해결할 수 있다.

부분문제 4

i 번째 지점에서 $N-i+1$ 이상으로 속력을 가진다면 도착 지점에서 0의 속력을 가질 수 없다. 그러므로 각 부분문제 3 풀이의 V_i 를 사용하는 부분에 $\min(V_i, N-i+1)$ 을 대신 사용해주면 문제를 해결할 수 있다. V 의 상한이 $K = 10^9$ 가 아닌 N 이 되므로 시간복잡도는 $O(N^2)$ 이다.

부분문제 5

i 번째 지점에서 가질 수 있는 최대 속력을 M_i 라고 하자. 이 값은 $M_{i+1} + 1$ 과 V_i 중 더 작은 값이다. 도착 지점을 속력 제한이 0인 $N+1$ 번째 지점이라고 생각하고 M_{N+1} 을 0으로 둔 뒤, N 부터 1까지 차례대로 M_i 을 채워주면, 모든 중간 지점에서의 최대 속력이 구해지므로 M 에 들어있는 값들의 합이 문제의 정답이 된다. 시간복잡도는 $O(N)$ 이다.

“호숫가의 개미굴” 문제 풀이

작성자: 나정휘

서술의 편의를 위해 개미굴의 방을 정점, 두 방을 연결하는 통로를 간선으로 표기한다. 즉, 문제에서 주어진 개미굴은 N 개의 정점으로 구성된 사이클, 그리고 사이클을 구성하는 각 정점에 C_i 개의 정점이 추가로 달려있는 그래프로 생각할 수 있다.

부분문제 1

다음과 같은 4가지 경우에 개미굴에 살 수 있는 개미의 최댓값을 모두 구한 뒤, 그 중 최댓값을 출력하면 된다.

- 1번 정점과 2번 정점에 모두 개미가 살지 않는 경우
- 1번 정점에 개미가 살지만 2번 정점에 개미가 살지 않는 경우
- 1번 정점에 개미가 살지 않지만 2번 정점에 개미가 사는 경우
- 1번 정점과 2번 정점에 모두 개미가 사는 경우

부분문제 2

N 개의 정점으로 구성된 단순 사이클만 있는 상황에서의 정답을 구해야 한다.

짝수 번째 정점에 개미를 한 마리씩 배치하면 $\lfloor \frac{N}{2} \rfloor$ 마리의 개미를 배치할 수 있고, 이보다 더 많이 배치할 수 없음을 알 수 있다. 따라서 $\lfloor \frac{N}{2} \rfloor$ 을 출력하면 된다.

부분문제 3

그래프 이론에서 각 간선이 연결하는 두 정점 중 최대 한 정점만 선택할 수 있을 때 정점을 최대한 많이 선택하는 문제를 **최대 독립 집합** 문제라고 부른다. 일반적인 그래프에서는 최대 독립 집합 문제를 다항 시간에 해결할 수 없지만, 트리에서는 동적 계획법을 이용해 $O(N)$ 시간에 해결할 수 있다.

구체적으로, 트리에서 아무 정점을 하나 잡아서 루트로 만든 다음 점화식을 다음과 같이 정의하면, 깊이 우선 탐색을 이용해 점화식을 계산해서 최대 독립 집합의 크기를 구할 수 있다.

- $D(v, 0)$: v 번 정점과 그의 자손 정점만 남긴 서브트리에서, v 번 정점을 선택하지 않았을 때의 최댓값
- $D(v, 1)$: v 번 정점과 그의 자손 정점만 남긴 서브트리에서, v 번 정점을 선택했을 때의 최댓값

사이클을 구성하는 간선 중 아무거나 하나를 끊으면 그래프가 트리로 바뀌고, 끊어진 간선 $e = (u, v)$ 가 연결하고 있는 두 정점 u, v 중 최대 하나만 선택할 수 있을 때의 최대 독립 집합을 구하면 된다. 따라서 이 문제는 트리 T 와 트리의 두 정점 u, v 가 주어졌을 때 u, v 중 최대 하나만 선택할 수 있는 최대 독립 집합 문제라고 생각할 수 있다.

부분문제 3은 사이클을 구성하는 모든 간선에 대해 동적 계획법을 수행하면 문제를 해결할 수 있다. 사이클을 구성하는 간선 $(i, i+1)$ 을 끊은 트리에서 i 번째 정점을 사용하지 않는 최대 독립 집합을 모든 간선에 대해 구하면, 그 중 최댓값이 원본 그래프의 최대 독립 집합의 크기가 된다. 이때 시간 복잡도는 $O((N + \sum C_i)^2)$ 이다.

부분문제 5

부분문제 3의 풀이를 확장하면 부분문제 5를 해결할 수 있다. 부분문제 3에서는 총 N 개의 트리에서 각각 동적 계획법을 수행했지만, 부분문제 5에서는 단 한 번의 동적 계획법을 이용해 문제를 해결해야 한다.

1번 정점과 N 번 정점을 잇는 간선을 끊은 트리에서, 깊이 우선 탐색을 이용해 다음과 같이 정의된 점화식을 계산하면 $O(N + \sum C_i)$ 번의 연산으로 문제를 해결할 수 있다.

- $D(v, 0, 0)$: v 번 정점과 그의 자손 정점만 남긴 서브트리에서, v 번 정점을 선택하지 않고 N 번 정점도 선택하지 않았을 때의 최댓값
- $D(v, 1, 0)$: v 번 정점과 그의 자손 정점만 남긴 서브트리에서, v 번 정점을 선택하고 N 번 정점을 선택하지 않았을 때의 최댓값
- $D(v, 0, 1)$: v 번 정점과 그의 자손 정점만 남긴 서브트리에서, v 번 정점을 선택하지 않았을 때의 최댓값
- $D(v, 1, 1)$: v 번 정점과 그의 자손 정점만 남긴 서브트리에서, v 번 정점을 선택했을 때의 최댓값

부분문제 7

3개 이상의 정점으로 구성된 모든 연결 그래프에는 차수가 1인 모든 정점을 포함하는 최대 독립 집합이 존재하고, 이는 귀류법을 이용해 증명할 수 있다.

차수가 1인 모든 정점을 포함하는 최대 독립 집합이 없다고 가정하면, 최대 독립 집합에 포함되지 않은 차수가 1인 정점 v 가 존재할 것이다. v 와 인접한 유일한 정점 x 는 최대 독립 집합에 포함되어 있어야 하며, 이때 x 의 차수는 1보다 크다. 그러므로 x 대신 v 를 넣더라도 최대 독립 집합을 만들 수 있다.

따라서 $N + \sum C_i \geq 3$ 이면 항상 모든 쪽방을 포함하는 정답이 존재하고, 부분문제 7은 $N \geq 2, C_i \geq 1$ 을 만족하므로 $\sum C_i$ 를 출력하면 문제를 해결할 수 있다.

부분문제 8

부분문제 8은 부분문제 7의 관찰을 확장해서 해결할 수 있다.

먼저 쪽방이 하나도 없는 경우를 먼저 처리하자. 이는 부분문제 2의 상황과 동일하므로 $\lfloor \frac{N}{2} \rfloor$ 를 출력하면 된다.

이제 쪽방이 최소한 하나 이상 있는, $N + \sum C_i \geq 3$ 인 경우만 생각해도 된다. 부분문제 7의 관찰에 의해 모든 쪽방을 포함하는 답안이 존재하므로 $C_i \geq 1$ 인 모든 정점에 달려 있는 쪽방에 개미를 배치할 수 있다. 그러면 $C_i = 0$ 인 정점들이 남게 되는데, $C_i, C_{i+1}, C_{i+2}, \dots, C_{j-1}, C_j$ 가 모두 0이면 $C_i, C_{i+2}, C_{i+4}, \dots$ 와 같이 홀수 번째 정점에 개미를 배치하는 것이 최적임을 알 수 있다. 즉, C_i 가 0인 구간에서는 홀수 번째 자리에 개미를 배치하면 되므로, $\lfloor (\text{구간의 길이} + 1) / 2 \rfloor$ 를 정답에 더하면 된다. 시간 복잡도는 $O(N)$ 이다.

C, C++, Java에서 64비트 정수 타입이 아닌 32비트 정수 타입을 사용하면 부분문제 6만 맞게 되고, $C_i = 0$ 인 구간을 확인하는 부분을 비효율적으로 구현하면 부분문제 4만 맞게 된다.

“고기 파티” 문제 풀이

작성자: 김동현

부분문제 1

각 사람마다, 지금 불판에 남아 있는 각 고기에 대해 몇 개의 꼬치가 이 고기를 찌르는지 일일이 세고, 2개의 꼬치에 찔린 고기들의 맛의 합을 구한 뒤 1개 이상의 꼬치에 찔린 모든 고기를 제거해주면 된다. $O(NM)$ 시간에 문제를 해결할 수 있다.

부분문제 2

같은 (s_i, e_i) 를 가지는 고기를 하나로 묶어서 생각하면, $e_i - s_i \leq 5$ 이므로 각 꼬치마다 해당 꼬치가 찌를 수 있는 고기는 최대 15종류임을 알 수 있다. `std::map` 등의 자료 구조를 이용하여 각 꼬치마다 해당하는 고기들 중 아직 찔리지 않은 것을 모두 찾아서 뽑아낸 뒤 두 꼬치에 모두 찔리는 고기는 답에 반영해주면 된다.

부분문제 3

i 번째 고기의 구간은 $i + 1$ 번째 고기의 구간을 포함하므로, $i + 1$ 번째 고기가 어느 시점에 꼬치에 찔렸다면 i 번째 이전의 모든 고기도 같이 찔리게 된다. 즉, 임의의 순간에 고기가 1개 이상 남아있다면 어떤 k ($1 \leq k \leq N$)에 대해 $k, k + 1, \dots, N$ 번째 고기들이 남아있는 형태일 것이다.

따라서, 고기들을 입력 받은 순서대로 큐(Queue) 자료구조에 넣은 뒤, 각 사람이 들어올 때마다 큐의 맨 앞에 있는 고기가 1개 이상의 꼬치에 찔린다면 pop을 수행해주는 것을 불가능할 때까지 반복하면 된다. 한 번 빠진 고기는 더 이상 고려되지 않으므로 총 시간복잡도는 $O(N + M)$ 이다.

부분문제 4

$e_i - s_i = L$ 이라 하면, 각 고기가 차지하는 구간을 $[s_i, s_i + L]$ 로 나타낼 수 있다. 따라서, 각 고기의 구간에 대한 정보를 s_i 만 가지고도 나타낼 수 있다.

$a_j + 0.1$ 좌표에 찌른 꼬치는 $a_j - L + 1 \leq s_i \leq a_j$ 를 만족하는 고기를 찌르고, $b_j + 0.9$ 좌표에 찌른 꼬치는 $b_j - L + 1 \leq s_i \leq b_j$ 를 만족하는 고기를 찌른다. 따라서, 해당 범위에 속하는 고기들을 빠르게 찾아내고 제거할 수 있다면 문제를 효율적으로 해결할 수 있다.

`std::set` 등의 균형 이진 트리 기반의 자료 구조를 사용하자. $a_j - L + 1 \leq s_i$ 이면서 최소인 s_i 를 자료 구조에서 찾고 제거하는 것을 $O(\log N)$ 에 할 수 있으므로, 해당하는 고기가 없거나 $s_i > a_j$ 가 되기 전까지 반복하면 $O(\text{범위 내에 남아있는 고기 수} + 1) \times \log N$ 시간복잡도에 특정 꼬치에 찔린 모든 고기를 자료구조에서 뽑아낼 수 있다. 두 꼬치 각각에 대해 이를 수행한 뒤, 뽑아낸 고기들 중 두 꼬치에 모두 찔린 고기들을 답에 더해지면 된다.

한 번 자료구조에서 뽑은 고기는 다시 고려하지 않으므로, 총 시간복잡도는 $O((N + M) \log N)$ 이다.

부분문제 5

부분문제 4의 아이디어를 약간 확장하여, 일반적인 경우에 대해서도 특정 꼬치에 찔리는 고기를 $O(\text{찔린 고기 수} \times \log N)$ 정도의 시간에 모두 뽑아낼 수 있다. 매우 다양한 방법으로 해결이 가능하며, 이 중 두 가지를 소개한다.

첫 번째 방법은 구간의 최댓값과 그 위치를 같이 관리하는 세그먼트 트리를 이용한다. 우선 주어진 고기들을 s_i 가 증가하는 순서대로 정렬하자. 그리고 세그먼트 트리의 인덱스 i 에 해당하는 값을 e_i 로 초기화한다. 어떤 꼬치를 좌표 $X + 0.1$ 에 찌른다고 하자. 이 때 이 꼬치에 찔리는 고기는 $s_i \leq X$, $X + 1 \leq e_i$ 를 만족하는 고기들이다. ($X + 0.9$ 에 찌르는 경우에도 동일하다.)

u 를 $s_u \leq X$ 를 만족하는 최대 인덱스라 하자. 세그먼트 트리의 $[1, u]$ 구간에서 최댓값을 찾고, 그 값이 $X + 1$ 이상이라면 뽑아내는 것을 반복하자. 뽑아낼 때는 해당하는 고기의 인덱스가 k 일 때, 인덱스 k 에 해당하는 값을 -1 로 설정하면 된다. 이것을 최댓값이 $X + 1$ 미만이 될 때까지 반복하게 되면 $O((\text{찔린 고기 수} + 1) \times \log N)$ 시간만에 특정 꼬치가 찌르는 모든 고기를 뽑을 수 있다. 총 시간복잡도는 부분문제 4와 마찬가지로 $O((N + M) \log N)$ 이다.

두 번째 방법은 세그먼트 트리의 각 노드마다 리스트를 관리하는 것이다. 각 고기가 차지하는 구간들의 양 끝점을 가지고 좌표 압축을 수행하면, 각 고기가 압축된 좌표에서 차지하는 구간을 세그먼트 트리 상에서 $O(\log N)$ 개의 노드로 쪼갤 수 있으므로 해당하는 노드들의 리스트에 각각 고기 번호를 삽입한다. 이렇게 하면 고기 번호를 자료구조에 총 $O(N \log N)$ 개만큼 넣게 된다.

이제 어떤 꼬치가 찌르는 고기를 모두 뽑아내기 위해서는, 해당 꼬치의 (압축된) 좌표를 포함하는 $O(\log N)$ 개의 노드들의 리스트에 들어 있는 고기 번호들을 모두 뽑아낸 뒤, 이전에 찔린 적이 없는 고기들만 취하면 된다. 한 번 번호를 뽑아낸 리스트는 빈 리스트가 되므로 뽑아내는 고기 번호는 많아야 $O(N \log N)$ 개이다. 고기 번호를 뽑아내는 것은 리스트 순회이고, 이전에 찔린 적이 없는지 체크하는 것은 boolean 배열로 가능하므로 고기 번호 하나당 $O(1)$ 만에 모든 필요한 처리가 가능하다. 총 시간복잡도는 $O((N + M) \log N)$ 이다.

“지그재그” 문제 풀이

작성자: 김준원

부분문제 1

가장 단순한 방법으로 문제를 푼다. $f(x, y, z)$ 를 구하기 위해, 수열 A 의 가능한 모든 부분수열을 직접 살펴보면서 조건 1., 2., 3.을 만족하는지 확인한다. 길이 N 인 수열 A 의 부분수열은 총 2^N 가지 존재한다(각 원소에 대해서 2가지 선택지 - ‘고른다’와 ‘고르지 않는다’가 도착하므로). 각각에 대해 반복문을 한 번 이용해 $O(N)$ 으로 세 조건을 모두 판별할 수 있다. 세 조건을 모두 만족하는 부분수열 중 가장 긴 부분수열의 길이가 $f(x, y, z)$ 가 될 것이다. 함수 f 의 $O(N^3)$ 가지 값을 모두 구해야 하므로, 총 시간 복잡도는 $O(N^4 2^N)$ 이다.

어떤 수열이 조건 1., 2.를 만족하는지는 쉽게 판별할 수 있으므로, 두 조건을 만족하는 부분수열만을 순회해서 조건 3.을 검사하는 방법도 있을 것이다.

부분문제 2

부분문제 1에서는 총 2^N 가지의 부분수열을 일일이 순회했다. 여기서 병목은 조건 3.이었는데, ‘어떤 수열의 가장 긴 지그재그 부분수열’을 더 빠르게 구할 수 있는 방법이 존재한다면 시간 복잡도를 크게 개선할 수 있을 것이다.

길이 K 인 수열 $S_1, \dots, S_K$ 의 가장 긴 지그재그 부분수열은 다음과 같은 간단한 그리디 알고리즘을 이용해 구할 수 있다.

1. 가장 긴 지그재그 부분수열을 담을 배열 arr 를 준비한다. 최초에 arr 은 빈 배열이다.
2. $i = 1, 2, \dots, K$ 를 순서대로 순회하며,
 - (a) arr 의 끝에 S_i 를 추가한 뒤에도 arr 이 지그재그 수열이라면, arr 의 끝에 S_i 를 추가한다.
 - (b) 그렇지 않다면, arr 의 마지막 원소를 S_i 로 대체한다.

이 알고리즘의 정당성은 직관적으로도 눈에 보이지만, 아래 부분문제 4에 대한 문단에서 구체적으로 논의할 것이다.

이로써 각 x, y, z 에 대해 $f(x, y, z)$ 를 $O(N)$ 의 시간 복잡도에 구할 수 있다. 먼저 조건 1., 2.를 필터링한 다음 위 알고리즘을 이용하면, 지그재그 수열이어야 한다는 조건 (조건 3.)까지 만족하는 가장 긴 부분수열을 구할 수 있다. 함수 f 의 $O(N^3)$ 가지의 값을 모두 구한 뒤 문제의 답을 구할 수 있으므로, 총 시간 복잡도는 $O(N^4)$ 이다.

부분문제 3

부분문제 2의 풀이를 개선하여 부분문제 3을 풀 수 있다. 부분문제 2의 풀이에서는 매번 비슷하게 계산되는 $O(N^3)$ 가지의 값을 구하고 있다는 것을 관찰할 수 있다. 슬라이딩 윈도우 기법을 사용하여 중복되는 계산들을 최적화할 수 있으며, $f(x, y, y+1), f(x, y, y+2), \dots, f(x, y, N)$ 까지의 값들을 한 번의 반복으로 구할 수 있다. 시간 복잡도가 $O(N^3)$ 이 되어 부분문제 3을 해결할 수 있다.

부분문제 4

부분문제 4부터의 부분문제들을 해결하려면 부분문제 3까지와 다른 관점에서 문제를 바라보아야 한다. 각 f 의 값들을 일일이 계산한 뒤 합쳐서 g 의 값을 만들지 말고, g 의 값을 직접 계산해보자. 여기에는 더 많은 관찰이 필요하다.

부분문제 2에서 제시한 가장 긴 지그재그 부분수열을 계산하는 알고리즘을 관찰하면, 수열 $S_1, \dots, S_K$ 의 가장 긴 지그재그 부분수열의 길이를 계산하는 간단한 공식을 알 수 있다. (a)가 실행될 때마다 arr 의 길이가 하나씩 증가한다. 다른 말로, 총 반복 횟수 K 에서 (b)가 실행되는 횟수를 빼면 알고리즘이 끝난 뒤 arr 가 된다. 즉, $A_y, \dots, A_z$ 에서 x 이하인 수들을 모아서 만든 수열을 C 라 하면, $f(x, y, z)$ 의 값은

1. C 의 길이
2. $C_i < C_{i+1} < C_{i+2}$ 를 만족하는 i 의 개수
3. $C_i > C_{i+1} > C_{i+2}$ 를 만족하는 i 의 개수

일 때 1. - 2. - 3.가 된다. $f(x, y, z)$ 의 합으로 정의되는 $g(x)$ 도 1.를 모두 합한 값에서 2., 3.를 모두 합한 값을 빼서 계산할 수 있다. 이 때, 고정된 x 에 대해 가능한 y, z 의 값을 모두 고려해 계산한다는 점에 유의하라. 즉, 위에서 정의한 부분수열 C 는, 모두 A 에서 x 이하인 수들을 모은 부분수열을 잘라 만든 수열이다. A 에서 x 이하인 수를 모은 배열을 $B_1, \dots, B_K$ 로 두고, 각 원소 B_j 의 배열 A 에서의 위치를 P_j 로 두자.

$g(x)$ 와 같은 값을 계산할 때에 유용하게 쓰이는 계산법은, 수열의 각 원소에 대해 해당 원소가 세어지는 횟수를 합하는 것이다. 각 j 에 대해 B_j 를 몇 번 세게 되는지 계산하자. 먼저, B_j 를 1.에서 세는 경우는 $y \leq P_j \leq z$ 인 경우이며, 총 $P_j \times (N - P_j + 1)$ 번이다. 2.에서 세는지 여부는 $B_j < B_{j+1} < B_{j+2}$ 인지에 달려 있다. 이 조건이 성립한다면, B_j 를 $P_j \times (N - P_{j+2} + 1)$ 번 세고, 그렇지 않다면 2.에서는 B_j 를 세지 않는다. 3.에서도 $B_j > B_{j+1} > B_{j+2}$ 가 성립한다면 $P_j \times (N - P_{j+2} + 1)$ 번 세고, 그렇지 않으면 세지 않는다.

위를 각각 계산한 다음 1. - 2. - 3.를 하면 $g(x)$ 에서 B_j 의 ‘기여도’가 된다. $B_1, B_2, \dots, B_K$ 에 대해 이 값들을 모두 합하면 $g(x)$ 의 값을 얻을 수 있다. 이를 별도의 최적화를 거치지 않고 구현하면 $O(N^3)$ 의 시간 복잡도로 부분문제 3까지 해결할 수 있고, 집합 자료구조 (예를 들어 C++의 `std::set`)을 이용해 구현하면 $O(N^2 \log N)$ 의 시간 복잡도로 부분문제 4까지 해결할 수 있다.

부분문제 5

부분문제 4와 같이 각 원소의 ‘기여도’를 세는 접근법은 이 풀이를 선형으로 개선할 때에도 그대로 써먹을 수 있다.

x 의 값이 $1, 2, \dots, N$ 일 때 $g(x)$ 의 값을 순서대로 계산하는 과정에서, x 의 값이 1 증가해서 $x+1$ 이 되는 순간을 일종의 쿼리로 생각할 수 있다. x 가 증가하여 $x+1$ 이 되는 것은 수열 B 의 어딘가에 $x+1$ 이 하나 추가된 것과 같다. 따라서 $g(x+1)$ 의 값을 계산할 때에는, $g(x)$ 의 값에서 새로 추가된 $x+1$ 의 ‘기여도’를 합해주면 된다. 이는 부분문제 4에서 소개한 방법으로 계산할 수 있다. 수열의 가운데에 $x+1$ 이 추가됨으로써 (추가된 위치 주변의) 다른 원소의 ‘기여도’가 변화하는데, 이 또한 계산에 포함시켜야 한다는 점 잊지 말자.

위 아이디어를 이용해 부분문제 4를 푸는 시간 복잡도에서 N 을 하나 제거할 수 있으며, $O(N \log N)$ 의 시간 복잡도로 전체 문제를 해결할 수 있다.